

prog_datasci_3_apython

September 11, 2022

1 Fonaments de Programació

1.1 Unitat 3: Estructures de control i funcions en Python

2 Instruccions d'ús

Aquest document és un *notebook* interactiu que intercala explicacions més aviat teòriques de conceptes de programació amb fragments de codi executables. Per aprofitar els avantatges que aporta aquest format, us recomanem que, en primer lloc, llegiu les explicacions i el codi que us proporcionem. D'aquesta manera, tindreu un primer contacte amb els conceptes que hi exposem. Ara bé, **la lectura és només el principi!** Una vegada hàgiu llegit el contingut, no oblideu executar el codi proporcionat i modificar-lo per crear-ne variants que us permetin comprovar que n'heu entès la funcionalitat i explorar-ne els detalls d'implementació. En darrer lloc, us recomanem també consultar la documentació enllaçada per explorar amb més profunditat les funcionalitats dels mòduls presentats.

Per guardar possibles modificacions que feu sobre aquest notebook, us aconsellem que munteu la unitat de Drive a Google Colaboratory (colab). Heu d'executar les instruccions següents:

```
[ ]: from google.colab import drive
     drive.mount('/content/drive')
```

```
[ ]: %cd /content/drive/MyDrive/Colab_Notebooks/prog_datasci_3
```

3 Introducció

Aquesta unitat continua presentant conceptes bàsics de programació en Python. En concret, presentarem les operacions lògiques i veurem com es pot alterar el flux d'execució dels programes amb estructures iteratives (*for* i *while*), condicionals (*if*) i funcions. Addicionalment, explicarem com

podem interactuar amb fitxers des de Python, i com podem importar altres mòduls per incorporar funcionalitats addicionals als nostres programes.

A continuació, s'inclou la taula de continguts. Per navegar pel document feu servir **Table of contents** (part superior esquerra) de google colab.

1. ¿Què és una estructura de control?
2. Condicional
 - 2.1 If
 - 2.2 If...else
 - 2.3 If...elif...else
3. Iteració
 - 3.1 For
 - 3.2 While
 - 3.3 Break i Continue
 - 3.4 Eines de Python per iterar eficientment
 - 3.4.1 Range
 - 3.4.2 Enumerate
 - 3.4.3 Zip
4. List Comprehensions
5. Funcions
 - 5.1 Args i kargs
 - 5.2 Funció Lambda
6. Llegir i escriure des de fitxers
 - 6.1 Pandas
7. Errors i Excepcions
 - 7.1 Handling Exceptions
 - 7.2 Raising Exceptions
8. Instruccions importants
9. Bibliografia

4 1 ¿Què és una estructura de control?

Els programes en Python executen les instruccions seqüencialment. Cada instrucció s'executa de manera ordenada i en el mateix ordre que estan escrites, com hem vist a la Unitat 2. A l'exemple següent es pot apreciar com s'executen les instruccions seqüencialment.

```
[1]: x = 3
      y = x + 7
      print("El valor de x és {}".format(x))
      print("El valor de y és {}".format(y))
```

El valor de x és 3
El valor de y és 10

Primer assignem la variable x igual a 3 i la variable y com la suma de x més el valor numèric 7. Tot seguit, mostrem per pantalla els valors de x i y . Cada instrucció s'executa una darrere l'altra. El flux d'execució d'un conjunt d'instruccions pot ser modificat mitjançant les estructures de control. Les principals estructures de control són les estructures de control condicionals (**if-elif-else**) i les estructures de control iteratives (**for** i **while**).

5 2 Condicional

L'estructura de control condicional és aquella estructura de control que permet implementar accions segons l'avaluació d'una condició simple, ja sigui falsa o veritable.

5.1 2.1 If

La instrucció **if** ens permet executar un bloc de codi si es compleix una determinada condició.

```
[2]: a = 10
      if a >= 5:
          print('a és més gran o igual a 5')
```

a és més gran o igual a 5

L'objectiu del codi anterior és avaluar si la variable a és més gran o igual a 5. Si la variable a és més gran o igual a 5, es compleix la condició de l'**if**, de manera que s'imprimeix per pantalla la instrucció **a és més gran o igual a 5**. En canvi, a la cel·la següent a és més petit que 5. Per tant, la condició de l'**if** no es compleix, i el bloc de codi de dins l'**if** no s'executa.

```
[3]: a = 2
      if a >= 5:
          print('a és més gran o igual a 5')
```

En els dos exemples anteriors podem apreciar l'estructura inherent de l'**if**. Aquesta estructura té dues parts ben diferenciades:

- **Condició** que s'ha de complir perquè el bloc de codi s'executi, en l'exemple anterior **a >= 5**.
- **Bloc de codi** que s'ha d'executar si es compleix la condició anterior. És important remarcar que el bloc de codi sempre ha de contenir una instrucció, si no així es genera un error de sintaxi **SyntaxError**.

La instrucció **if** s'ha d'acabar amb **:** i el bloc de codi ha d'estar tabulat. Tota sentència inserida després de l'**if** i correctament tabulada, forma part del bloc de codi que s'executarà si la condició

es compleix. A l'exemple següent, la instrucció `print('Fora if')` s'està executant sempre perquè no forma part del bloc de codi de l'`if`.

```
[4]: a = 2
     if a >= 5:
         print('a és més gran o igual a 5')
     print('Fora if')
```

Fora if

5.2 2.2 if ... else

Si la condició no es compleix, es pot executar un segon bloc de codi especificat dins la instrucció `else`.

```
[5]: a = 9
     if a % 2 == 0:
         print('a és parell')
     else:
         print('a no és parell')
```

a no és parell

La clàusula `else` especifica què cal fer si la condició no es compleix. En aquest exemple, la primera condició no es compleix perquè a no és parell, per tant, s'executa la instrucció que hi ha dins d'`else`.

5.3 2.3 If...elif...else

Si volem executar condicions addicionals podem fer servir la instrucció `elif`.

```
[6]: a = 9
     if a % 2 == 0:
         print('a és parell')
     elif a % 3 == 0:
         print('a és múltiple de 3')
```

a és múltiple de 3

En l'exemple anterior, si es compleix la primera condició, s'imprimeix per pantalla **a és parell**. Si es compleix la segona condició, s'imprimeix per pantalla **a és múltiple de 3**. També podem incloure clàusules `else` dins de l'estructura `if-elif` si no es compleix cap de les condicions anteriors.

```
[7]: a = 7
     if a % 2 == 0:
         print('a és parell')
     elif a % 3 == 0:
         print('a és múltiple de 3')
     else:
```

```
print('a no és parell ni múltiple 3')
```

a no és parell ni múltiple 3

Vegem-ne uns quants exemples més per veure diferents estructures condicionals. Us aconsellem que varieu els valors de les variables de codi dels exemples següents i aneu executant el codi per comprovar com es comporta en cada situació.

```
[8]: # Exemple 1
a = 5
b = 5
if a > b:
    print('a és més gran que b')
elif a < b:
    print('a és més petit que b')
else:
    print('a és igual a b')
```

a és igual a b

```
[9]: # Exemple 2
a = -1
b = -2
if a > b:
    if b >= 0:
        print('a es més gran que b i positiu')
    elif a*-1 > 0:
        print('a és més gran que b i negatiu')
    else:
        print('a és més gran que b i positiu')
else:
    print('b és més gran o igual que a')
```

a és més gran que b i negatiu

6 3 Iteració

Les estructures de control iteratives permeten executar un mateix bloc de codi tantes vegades com sigui necessari. En la majoria de llenguatges de programació, hi ha dues maneres d'iterar una seqüència: mitjançant `for` o mitjançant `while`.

6.1 3.1 For

La instrucció `for` ens permet crear bucles sobre un nombre d'iteracions definit inicialment.

```
[10]: monsters = ['Kraken', 'Leviathan', 'Uroborus', 'Hydra']
```

```
# Primer mètode iterant mitjançant for:  
for monster in monsters:  
    print(monster)
```

Kraken
Leviathan
Uroborus
Hydra

En l'exemple anterior, estem recorrent cada element de la llista `monsters` i imprimint-lo per pantalla. L'estructura d'iteració `for` segueix l'estructura següent:

```
[28]: # for <variable> in <iterable>:  
      #     <codi>
```

on la `variable` pren cada un dels valors de l'iterable. `iterable` són aquells objectes que poden ser iterats o que poden ser indexats. Alguns exemples d'iterables són les llistes, tuples, cadenes o diccionaris. És important remarcar que tota sentència inserida després del `for` ha d'estar correctament tabulada perquè s'executi. Com podeu veure a la cel·la següent, el `print` no està tabulat i crea un error `IndentationError`.

```
[11]: for monster in monsters:  
      print(monster)
```

```
File "<ipython-input-11-baf8f65b9206>", line 2  
print(monster)  
^
```

IndentationError: expected an indented block

A l'exemple següent, veurem diferents tipus d'iterables.

```
[12]: # Definim una llista, una tupla, un diccionari i una cadena de caràcters  
planetes_llista = ['Mercuri', 'Venus', 'Terra']  
planetes_tupla = ('Mercuri', 'Venus', 'Terra')  
planetes_dict = {"Mercuri": 4880, "Venus": 12105, "Terra": 12745}  
planetes_str = 'Mercuri'  
  
# Recorrem les estructures amb un *for* i en mostrem el contingut  
print("**Llista**")  
for i in planetes_llista:  
    print(i, end=" ")  
print()  
  
print("**Tupla**")
```

```

for i in planetes_tupla:
    print(i, end=" ")
print()

print("**dict**")
for i in planetes_dict:
    print(i, end=" ")
print()

print("**cadena**")
for i in planetes_str:
    print(i, end=" ")

```

```

**Llista**
Mercuri Venus Terra
**Tupla**
Mercuri Venus Terra
**dict**
Mercuri Venus Terra
**cadena**
M e r c u r i

```

És important remarcar que un valor numèric no és un objecte iterable perquè no pot ser indexat. Si executem l'exemple següent s'obté l'error `TypeError: 'int' object is not iterable`.

```

[13]: a = 5

for i in a:
    print(a)

```

```

└─
└─
-----
TypeError                                Traceback (most recent call last)

<ipython-input-13-7938d9b1b2cb> in <module>
      1 a = 5
      2
----> 3 for i in a:
      4     print(a)

TypeError: 'int' object is not iterable

```

Si volem iterar una seqüència de valors numèrics, haurem de crear una llista de valors i iterar sobre aquesta llista.

```
[14]: # Primer creem la llista de valors numèrics del 0 fins al 5 (inclusius)
llista_valors = [0, 1, 2, 3, 4, 5]

# Recorrem la llista amb un *for* i en mostrem el contingut
for i in llista_valors:
    print(i, end=" ")
```

0 1 2 3 4 5

A la secció 3.5 veurem com podem crear una seqüència de valors numèrics mitjançant la funció `range()` sense haver de definir cap llista abans.

La instrucció `for` permet incloure altres estructures de control dins del seu bloc de codi, per exemple `for`. Això permet iterar un objecte que en cada element té un objecte iterable.

```
[15]: planetes_llista = ['Mercuri', 'Venus', 'Terra']

# Recorrem la llista amb un *for* i en mostrem el contingut

for i in planetes_llista:
    print(i, end=" ")
    # Recorrem cada element de la cadena i comptem el nombre de lletres que
    ↳formen la paraula
    print()
    k = 0
    for j in i:
        # S'actualitza el valor de k sumant-li 1
        k += 1
    print("La paraula {} està formada per {} lletres".format((i), (k)))
```

Mercuri

La paraula Mercuri està formada per 7 lletres

Venus

La paraula Venus està formada per 5 lletres

Terra

La paraula Terra està formada per 5 lletres

6.2 3.2 While

La instrucció `while` permet iterar un objecte mentre es compleixi una condició determinada. Quan la condició deixa de complir-se, se surt del bucle i continua l'execució del bloc de codi següent. A l'exemple tot seguit, el codi dins del `while` s'executarà mentre `a` sigui més gran que `b`.

```
[16]: a = 5
b = 0

while a > b:
    # S'actualitza el valor de b sumant-li 1
```



```
b += 1
print("b és igual {}".format(b))
```

b és igual 5

També es pot iterar una llista mitjançant `while`, però és una manera molt menys idiomàtica en Python i és preferible l'opció de `for`.

```
[17]: # Mentre l'índex 'i' sigui més petit que la longitud de la llista 'planetes':
i = 0
while i < len(planetes_llista):
    # Imprimeix el valor de la llista en la posició 'i'.
    print(i, planetes_llista[i])
    # No ens oblidem d'actualitzar el valor d''i' sumant-li 1 o tindrem un bucle
    ↪ infinit.
    i += 1
```

0 Mercuri
1 Venus
2 Terra

En aquest moment seríem capaços de calcular la sèrie de Fibonacci fins a un determinat valor:

```
[18]: # Calculem el valor de la sèrie fins a un valor n = 100.
n = 100

a, b = 0, 1
while a < n:
    print(a, end=" ")
    a, b = b, a+b
```

0 1 1 2 3 5 8 13 21 34 55 89

6.3 3.3 Break i Continue

Les instruccions `break` i `continue` ens permeten alterar el comportament de les instruccions `for` i `while`.

- **break**: permet sortir d'un bucle en un moment donat i aturar-ne l'execució.
- **continue**: permet saltar-se el codi restant en la iteració actual tornant al principi del bucle.

```
[19]: for planeta in planetes_llista:
    if planeta == "Venus":
        break
    print(planeta)
```

Mercuri

```
[20]: for planeta in planetes_llista:
        if planeta == "Venus":
            continue
        print(planeta)
```

Mercuri

Terra

En els exemples de les cel·les anteriors, es pot apreciar la diferència en el funcionament de les instruccions `break` i `continue`. En el primer exemple, `break` interromp l'execució del bucle quan *planeta* és igual a *Venus*. En canvi, `continue` no interromp el bucle, si no, passa a la iteració següent saltant la instrucció pendent `print`.

A l'exemple següent, canvieu la instrucció `break` per `continue`. Executeu el codi en les dues tessitures per comprovar com varia l'output final segons la instrucció que s'hagi fet servir.

```
[21]: a = 20
#
while a > 0:
    a -= 1
    if a % 2 == 0:
        continue # break
    print(a, end=" ")
```

19 17 15 13 11 9 7 5 3 1

6.4 3.4 Eines de Python per a iterar eficientment

Abans hem definit dues estructures de control, `for` i `while`, generals de la majoria de llenguatges de programació. Python té eines específiques per optimitzar el procés d'iteració. Entre aquestes eines hi ha `range()`, `enumerate()` i `zip()`.

6.4.1 3.4.1 Range

`Range` és una funció molt útil de Python per generar una seqüència de nombres. La sintaxi de `range` és la següent:

```
[27]: # range(start,end,step)
```

`Range` és una funció que permet passar tres paràmetres separats per coma, en què **start** és el valor inicial de la seqüència, **end** és el valor final i **step** el salt entre números. És molt important remarcar que `range` retorna una llista de nombres fins la posició final menys 1. Per defecte, el valor inicial és 0 i el salt és 1.

```
[22]: # La funció 'range' ens retorna una llista de nombres:
list(range(10))
```

```
[22]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Fixeu-vos que `range` no torna directament una llista, sinó que retorna un tipus propi del mateix nom. Per aquest motiu, per obtenir una llista cal fer una conversió de tipus fent servir `list()`. Això és una novetat en Python 3, ja que en Python 2 la funció `range` retornava una llista.

```
[23]: # Visualitzem el tipus de retorn de range
print(type(range(10)))
print(type(list(range(10))))
```

```
<class 'range'>
<class 'list'>
```

Vegem alguns dels usos més habituals de `range`:

```
[24]: # Podem fer-la servir per iterar:
for i in range(10):
    print(i, end=" ")
print()
```

```
0 1 2 3 4 5 6 7 8 9
```

```
[25]: # Podem definir el rang d'acció.

# Per exemple, especificant només el final com hem fet abans
for i in range(10):
    print(i, end=" ")

print()

# especificant inici i fi:
for i in range(5, 10):
    print(i, end=" ")

print()

# o especificant també el salt entre cada valor:
for i in range(5, 10, 3):
    print(i, end=" ")
```

```
0 1 2 3 4 5 6 7 8 9
5 6 7 8 9
5 8
```

Sempre que no necessitem explícitament una llista, farem servir directament el tipus `range` (sense fer la conversió a llista), ja que això és en general més eficient (estalvia memòria). Només quan necessitem una llista, (per exemple, per visualitzar el resultat com hem fet en el primer exemple) convertirem el resultat de `range` a llista.

```
[26]: # També és possible iterar en un diccionari:
pais_codis = {34: 'Spain', 376: 'Andorra', 41: 'Switzerland', 424: None}
```

```

# Per clau:
for pais_codi in pais_codis.keys():
    print(pais_codi)
print()

# Per valor:
for pais in pais_codis.values():
    print(pais)
print()

# Per tots dos alhora:
for pais_codi, pais in pais_codis.items():
    print(pais_codi, pais)

```

```

34
376
41
424

```

```

Spain
Andorra
Switzerland
None

```

```

34 Spain
376 Andorra
41 Switzerland
424 None

```

6.4.2 3.4.2 Enumerate

Enumerate és una funció de Python per accedir als índexs d'una col·lecció. Aquesta funció retorna una tupla en què el primer element és un índex que comença per 0 i augmenta d'1 a 1, i el segon element és el valor de la posició a la llista.

A l'exemple següent, accedim a l'índex i al valor de la llista mitjançant **for** i un comptador:

```

[27]: # Recuperem l'exemple dels monstres
monsters = ['Kraken', 'Leviathan', 'Uroborus', 'Hydra']

# Inicialitzem el comptador
i = 0

for monster in monsters:
    print(i, monster, end=" ")
    i += 1

```

0 Kraken 1 Leviathan 2 Uroborus 3 Hydra

Podem obtenir el mateix resultat fent servir la funció `enumerate`:

```
[28]: for index, monster in enumerate(monsters):  
       print(index, monster, end=" ")
```

0 Kraken 1 Leviathan 2 Uroborus 3 Hydra

6.4.3 3.4.3 Zip

`Zip` és una funció que, combinada amb un `for`, permet iterar tantes llistes com sigui necessari en paral·lel.

```
[29]: planetes = ["Mercuri", "Venus", "Terra"]  
       diamestre = ["4880 km", "12105 km", "12750 km"]  
       distancia_sol = ["57910000 km", "108200000 km", "146600000 km"]  
  
       for pla, dist, ds in zip(planetes, diamestre, distancia_sol):  
           print("{} té un diamestre de {} i es troba a una distància del Sol de {}".  
                 ↪format(  
                     (pla), (dist), (ds)))
```

Mercuri té un diamestre de 4880 km i es troba a una distància del Sol de 57910000 km

Venus té un diamestre de 12105 km i es troba a una distància del Sol de 108200000 km

Terra té un diamestre de 12750 km i es troba a una distància del Sol de 146600000 km

Si les llistes tenen longituds diferents, la iteració s'atura quan la llista més petita s'acaba.

```
[30]: planetes = ["Mercuri", "Venus", "Terra"]  
       diamestre = ["4880 km", "12105 km", "12750 km"]  
       distancia_sol = ["57910000 km", "108200000 km"]  
  
       for pla, dist, ds in zip(planetes, diamestre, distancia_sol):  
           print("{} té un diamestre de {} i es troba a una distància del Sol de {}".  
                 ↪format(  
                     (pla), (dist), (ds)))
```

Mercuri té un diamestre de 4880 km i es troba a una distància del Sol de 57910000 km

Venus té un diamestre de 12105 km i es troba a una distància del Sol de 108200000 km

Aquesta funció està definida per qualsevol objecte iterable. Aquesta característica fa que es pugui fer servir `zip` amb diccionaris. Per accedir al valor i la clau de cada element s'ha de fer servir la funció `items`.

```
[31]: pais_codi = {34: 'Spain', 376: 'Andorra', 41: 'Switzerland', 424: None}
pais_hab = {'Spain': 46000000, 'Andorra': 77000, 'Switzerland': 8000000}

for (a, b), (c, d) in zip(pais_codi.items(), pais_hab.items()):
    print(a, b, d)
```

```
34 Spain 46000000
376 Andorra 77000
41 Switzerland 8000000
```

7 4 List Comprehensions

List comprehension és una expressió idiomàtica de Python que permet crear llistes d'elements en una sola línia de codi de forma senzilla i elegant sense fer servir estructures de control molt complexes.

```
[32]: # Volem seleccionar tots aquells planetes que tenen una *r* en el seu nom
planetes = ["Mercuri", "Venus", "Terra"]
subset = []
for x in planetes:
    if "r" in x:
        subset.append(x)
print(subset)
```

```
['Mercuri', 'Terra']
```

En l'exemple anterior, hem implementat una estructura de control clàssica amb `for` i `if` per seleccionar els elements de la llista que tenen una *r* en el seu nom. A la cel·la següent farem el mateix fent servir *list comprehension*.

```
[33]: subset = [x for x in planetes if "r" in x]

print(subset)
```

```
['Mercuri', 'Terra']
```

La sintaxi general d'una *list comprehension* és la següent:

```
[115]: # llista = [expressió for element in iterable]
```

Aquesta sintaxi té dues parts ben diferenciades.

- `for element in iterable` iteració d'un determinat iterable i es guarda cada un dels elements a `element`. L'iterable pot ser una llista, seqüència, generador o una estructura de control (com per exemple, `if`) que retorni els seus elements d'un en un.
- `expressió` acció sobre `element` que s'afegirà a la llista a cada iteració.

```
[34]: # Aplicarem una list comprehension per calcular el quadrat dels nombres parells
      ↪ presents fins al 9
resultat = [i**2 for i in range(10) if i % 2 == 0]
print(resultat)
```

[0, 4, 16, 36, 64]

8 5 Funcions

Una altra manera molt important d'organitzar el flux d'execució és encapsulant una certa porció de codi en una funció reutilitzable. S'ha d'emfatitzar que el conjunt d'instruccions que s'executen dins la funció han d'estar correctament tabulades respecte de `def()`. Si no és així, s'obté un error de tabulació.

Una funció en Python utilitza un concepte similar al d'una funció matemàtica. Per exemple, imaginem-nos la funció matemàtica:

$$\text{suma}(x, y) = x + y$$

A Python podem definir la mateixa funció de la manera següent:

```
[35]: # La funció 'suma' es defineix mitjançant la paraula especial 'def' i té dos
      ↪ arguments: 'x' i 'y':
def suma(x, y):
    # Mostrem per pantalla els valors de x i y
    print("El valor d'x és {}".format(x))
    print("El valor d'y és {}".format(y))
    # Tornem el valor de la suma
    return x + y
```

La definició d'una funció Python té les característiques següents:

- Paraula clau `def()`.
- Nom de la funció.
- Parèntesis i dins dels parèntesis els paràmetres d'entrada (tot i que poden ser opcionals).
- Dos punts `:`.
- Bloc de codi.
- Sentència de retorn, `return`, del resultat (opcional).

En la definició d'una funció no s'executa el bloc de codi que conté la funció. Per executar-lo, s'ha d'escriure el nom de la funció amb els paràmetres d'entrada corresponents, com podem veure als exemples següents. En el primer exemple, estem executant la funció `suma()` fent servir el nom dels paràmetres.

```
[36]: # Execució de la funció suma
suma(x=5, y=-5)
```

El valor d'x és 5
El valor d'y és -5

[36]: 0

Si fem servir el nom dels paràmetres, podem canviar l'ordre dels paràmetres sense modificar el resultat de la funció.

```
[37]: # Execució de la funció suma canviant l'ordre dels paràmetres
suma(y=-5, x=5)
```

El valor d'x és 5
El valor d'y és -5

[37]: 0

Si no fem servir el nom dels paràmetres, cada posició correspon a un paràmetre tal com hem definit dins de la funció. En el nostre cas, la primera posició és x, i la segona, y.

```
[38]: # Execució de la funció amb els arguments posicionals
suma(4, 2)
```

El valor d'x és 4
El valor d'y és 2

[38]: 6

```
[39]: # Execució de la funció canviant l'ordre els arguments posicionals
suma(2, 4)
```

El valor d'x és 2
El valor d'y és 4

[39]: 6

```
[40]: # Podem definir una funció que no faci res fent servir la paraula especial
↳ 'pass':

def dummy():
    pass
```

```
[41]: dummy()
```

```
[42]: # Podríem tornar a definir el tros de codi de la seqüència de Fibonacci com una
↳ funció:

def fibonacci(n=100):
    a, b = 0, 1
    while a < n:
        print(a, end=" ")
        a, b = b, a+b
```



```
[43]: fibonacci(10)
```

```
0 1 1 2 3 5 8
```

```
[44]: # A l'exemple anterior, hem definit que l'argument n tingui un valor per defecte.  
      ↪ Això és molt útil  
      # en els casos en què fem servir la funció sempre amb un mateix valor, vulguem  
      ↪ deixar constància d'un  
      # cas d'exemple o per defecte. En aquest cas, podem executar la funció sense  
      ↪ passar-li cap valor:  
      fibonacci()
```

```
0 1 1 2 3 5 8 13 21 34 55 89
```

```
[45]: # Podem definir part dels arguments amb valors per defecte i un altre sense.  
      # Els arguments sense valor per defecte sempre han d'estar més a l'esquerra a la  
      ↪ definició de la funció:  
  
      def potencia(a, b=2):  
          # Per defecte, elevarem al quadrat.  
          return a**b
```

```
[46]: print(potencia(3))  
      print(potencia(2, 3))
```

```
9
```

```
8
```

8.1 5.1 args i kargs

Tal com hem vist a l'apartat anterior, hem definit les funcions considerant un nombre concret d'arguments ($0, 1, 2, \dots, N$). Però ens podem trobar que, a priori, no sapiguem quantes variables necessitem. Per solucionar això, en Python es pot definir un conjunt d'arguments variable amb `args` i `kargs`.

Args ens permet definir funcions genèriques que acceptin un nombre variable d'arguments.

```
[47]: def suma_args(*args):  
      # Tornem el valor de la suma  
      return sum(args)
```

```
[49]: suma_args(3, 4, 5)
```

```
[49]: 12
```

La funció `suma_args` correspon a la mateixa funció `suma`, que hem definit a l'apartat anterior, però fent servir un nombre variable d'arguments. Si comparem les dues funcions, veurem que la funció

suma només permet sumar dos nombres. En canvi, `suma_args` permet sumar tants nombres com vulgui l'usuari.

```
[50]: print("Suma amb un nombre variable d'arguments és {}".format(suma_args(4, 5)))  
      print("Suma de dos arguments és {}".format(suma(4, 5)))
```

```
Suma amb un nombre variable d'arguments és 9  
El valor d'x és 4  
El valor d'y és 5  
Suma de dos arguments és 9
```

```
[51]: print("Suma amb un nombre variable d'arguments és {}".format(  
      suma_args(4, 5, 7, 8, 9)))  
      #print("Suma de dos arguments és {}".format(suma(4,5,7,8,9)))
```

```
Suma amb un nombre variable d'arguments és 33
```

Si es vol sumar 5 nombres mitjançant la funció `suma` definida amb dues variables, l'execució dona error perquè estàs fent servir més arguments dels definits.

Si, a més de treballar amb un nombre variable d'arguments, volem tenir la capacitat de definir el nom de cada uns dels arguments d'entrada, hem de fer servir **kargs**. Hem de treballar **kargs** com si es tractés d'un diccionari i accedir als corresponents valors i claus mitjançant la funció `items`.

```
[52]: def suma_kargs(**kargs):  
      r = 0  
      # Tornem el valor de la suma  
      for k, v in kargs.items():  
          print(k, "=", v)  
          r += v  
      return r
```

```
[53]: print("Suma amb un nombre variable d'arguments és {}".format(  
      suma_kargs(a=4, b=5, c=7, d=8, e=9)))
```

```
a = 4  
b = 5  
c = 7  
d = 8  
e = 9  
Suma amb un nombre variable d'arguments és 33
```

8.2 5.2 Funció Lambda

La funció *Lambda* o anònima és un tipus de funció de Python que s'empra per simplificar l'escriptura de funcions en una sola línia. A l'exemple següent volem obtenir tots els nombres múltiples de 5.

```
[54]: # Definim la funció myfun(), en què el paràmetre d'entrada és el valor fins on
      ↪ volem estudiar

def myfun(xmax):
    # Definim la llista
    mylist = []
    for k in range(0, xmax):
        # Comprobem si el valor és multiple de 5
        if k % 5 == 0:
            # Afegim el valor a la llista amb append()
            mylist.append(k)
    # Mostrem per pantalla
    print(mylist)
```

```
[55]: myfun(101)
```

```
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95,
100]
```

Podem expressar la funció `myfun()` fent servir la funció *Lambda* com:

```
[56]: # La funció filter() permet extreure els valors que compleixen una determina
      ↪ condició
print(list(filter(lambda x: x % 5 == 0, range(0, 101))))
```

```
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95,
100]
```

Com es pot veure en l'exemple anterior, la funció `lambda` (en combinació amb la funció `filter`) ens permet obtenir el mateix resultat que la funció `myfun` de manera més simplificada i compacta.

9 6 LLegir i escriure des de fitxers

Una tasca habitual és llegir línies d'un fitxer o escriure línies en un fitxer. A continuació, us expliquem com podem escriure i llegir un fitxer:

```
[55]: # Primer de tot, escriurem en un fitxer:

# Obrim un fitxer de nom 'a_file.txt' per a escriptura (d'aquí la 'w',
      ↪ 'writing').
# L'assignem a un objecte per gestionar el fitxer de nom 'out':
out = open('a_file.txt', 'w')
# Escriurem 10 línies, cadascuna amb un número del 0 al 9.
for i in range(10):
    # La línia següent escriu al fitxer tot el que hi posem dins 'out.write()'
    # En el nostre cas, és un string del tipus '0\n', '1\n', etc. Això ho
      ↪ aconseguim
```

```

# fent servir format amb un paràmetre, que és el número de línia.
# Afegim el \n al final de cada línia, per incloure el salt de línia
out.write("Línia {}".format(i))

# Alternativament, podríem fer servir els wildcards %d i %s.
# %s representa un string o una cadena de caràcters i %d un nombre enter.
# Concatenem en aquest cas un nombre amb un string que es tracta del salt de
# línia, os.linesep, que equival a Linux
# a '\n':
# out.write("Línia %d%s" % (i, os.linesep))

# Un cop hem acabat d'escriure el fitxer, s'ha de tancar amb la instrucció
→close()
out.close()

```

Ara llegirem el fitxer que acabem d'escriure de tres maneres diferents. En els dos primers mètodes, un cop hem acabat de treballar amb l'arxiu, s'ha de tancar amb la instrucció `close()`. En canvi, el tercer mètode, `with()`, tanca l'arxiu automàticament.

```

[56]: # Primer mètode
f = open('a_file.txt')
for line in f:
    print(line, end="")
f.close()
print()

# Segon mètode
f = open('a_file.txt')
lines = f.readlines()
for line in lines:
    print(line, end="")
f.close()
print()

# Tercer mètode
with open('a_file.txt') as f:
    for line in f:
        print(line, end="")

```

```

Línia 0
Línia 1
Línia 2
Línia 3
Línia 4
Línia 5
Línia 6
Línia 7

```

Línia 8
Línia 9

Línia 0
Línia 1
Línia 2
Línia 3
Línia 4
Línia 5
Línia 6
Línia 7
Línia 8
Línia 9

Línia 0
Línia 1
Línia 2
Línia 3
Línia 4
Línia 5
Línia 6
Línia 7
Línia 8
Línia 9

9.1 6.1 Pandas

Una altra manera de llegir i escriure fitxers en Python és mitjançant la llibreria *Pandas*. Per poder fer ús de les funcions d'una llibreria, primer de tot s'ha importar la llibreria mitjançant la instrucció *import*.

```
[58]: import pandas as pd
```

Pandas ens permet carregar les dades d'un fitxer *CSV* directament a un dataframe per mitjà de la funció `read_csv`. Aquesta funció és molt versàtil i disposa de molts paràmetres per configurar amb tot detall com dur a terme la importació. En molts casos, la configuració per defecte ja ens oferirà els resultats desitjats.

Ara carregarem les dades del fitxer `marvel-wiki-data.csv`, que conté dades sobre personatges de còmic de Marvel. El conjunt de dades va ser creat pel web [FiveThirtyEight](#), que escriu articles basats en dades sobre esports i notícies, i que posa a disposició pública els [conjunts de dades](#) que recull per als seus articles.

```
[59]: data = pd.read_csv("data/marvel-wiki-data.csv")  
      print(type(data))
```

```
<class 'pandas.core.frame.DataFrame'>
```

D'una manera anàloga a com hem carregat les dades d'un fitxer a un *dataframe*, podem escriure les dades d'un *dataframe* a un fitxer CSV.

```
[60]: # Seleccionem les primeres 20 línies del dataframe data
new_data = data.head(n=20)
# Guardem el dataframe new_data en un CSV
new_data.to_csv("data/marvel-wiki-data-head20.csv", encoding='utf-8')
```

A la unitat 4 veurem més funcionalitats de la llibreria pandas i presentarem amb més detall els tipus de dades que retorna `read_csv`.

10 7 Errors i Excepcions

La codificació està subjecta a errors. Conèixer i corregir aquests errors forma part de l'aprenentatge de qualsevol llenguatge de programació. En Python, hi ha dues menes diferents d'errors: errors de sintaxi i Excepcions. La principal diferència entre les dues menes d'errors és que el primer està associat a un error de sintaxi, i el segon, a un error d'execució.

- **Errors de Sintaxi o d'interpretació:** Són errors provocats per un error en la codificació. Quan hi ha un error d'aquest tipus, l'interpret reproduïx la línia responsable de l'error i mostra amb una fletxa on hi ha hagut l'error.

```
[61]: i = 0
while i == 0
    print("hola")
    i += 1
```

```
File "<ipython-input-61-92300af304f3>", line 2
while i == 0
    ^
SyntaxError: invalid syntax
```

L'error ens està indicant que hi ha hagut un error de sintaxi a la segona línia en la instrucció `while`. En aquest cas, falten dos punts (':') al final.

```
[62]: i = 0
while i == 0:
    print("hola")
    i += 1
```

hola

- **Excepcions:** Els errors detectats durant l'execució, tot i que la sintaxi sigui correcta, s'anomenen **excepcions**. La majoria d'excepcions donen un missatge d'error. L'exemple següent mostra un error d'execució:

```
[63]: a = 10
      b = 0
      print(a/b)
```

```

-----
ZeroDivisionError                                Traceback (most recent call last)

<ipython-input-63-0ca4df5e1f05> in <module>
      1 a = 10
      2 b = 0
----> 3 print(a/b)

ZeroDivisionError: division by zero
```

L'última línia del missatge d'error informa d'allò que passa. Hi ha diferents tipus d'excepcions, les principals són **ZeroDivisionError**, **NameError**, **TypeError**. A les cel·les següents veurem un exemple de les excepcions **NameError** i **TypeError**.

```
[64]: # Excepció NameError
      x = 3 + parametre
```

```

-----
NameError                                Traceback (most recent call last)

<ipython-input-64-392d838d537f> in <module>
      1 # Excepció NameError
----> 2 x = 3 + parametre

NameError: name 'parametre' is not defined
```

L'excepció **NameError** es produeix perquè estem executant una ordre, **suma**, amb una variable que no està definida. En canvi, l'error següent, **TypeError** s'origina perquè estem intentant sumar dos tipus de magnituds que no es poden sumar, nombre enter amb una cadena de caràcters.

```
[65]: # Excepció TypeError
      x = 3 + '2'
```

↳

```
TypeError                                Traceback (most recent call last)

<ipython-input-65-f6730bb87b1a> in <module>
      1 # Excepció TypeError
----> 2 x = 3 + '2'

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

10.1 7.1 Handling Exceptions

La gestió de les excepcions és molt important per al bon funcionament del programa. Si no és així, l'aparició d'una excepció pot provocar que el programa s'aturi. Les excepcions poden ser capturades i gestionades adequadament, sense que el programa s'aturi fent ús de les instruccions `try` i `except`. Si recuperem l'exemple anterior:

```
[65]: a = 10
      b = 0
      while b < 5:
          try:
              c = a/b
              print(c)
              break
          except ZeroDivisionError:
              b += 1
              print("No es pot fer la divisió")
```

```
No es pot fer la divisió
10.0
```

En aquest exemple, el programa no s'atura i pot continuar executant-se. Quan hi ha una *excepció* s'executa el codi que hi ha dins de la instrucció `except`, però el programa no s'atura. La instrucció `except` també permet fer servir excepcions genèriques mitjançant `Exception` quan es desconeix l'excepció que es crea.

```
[66]: a = 10
      b = 0
      while b < 5:
          try:
              c = a/b
              print(c)
              break
          except Exception:
              b += 1
```



```
print("No es pot fer la divisió")
```

No es pot fer la divisió
10.0

Altres instruccions per afegir als blocs de codi `try` `except` `else` `finally`.

- `else`: S'executa si no hi ha hagut cap excepció.
- `finally`: S'executa sempre encara que hi hagi hagut una excepció.

```
[67]: a = 10
      b = 0
      while b < 5:
          try:
              c = a/b
          except ZeroDivisionError:
              b += 1
              print("No es pot fer la divisió")
          else:
              print("S'ha executat la divisió i el resultat ha estat {}".format(c))
              break
```

No es pot fer la divisió
S'ha executat la divisió i el resultat ha estat 10.0

```
[68]: a = 10
      b = 1
      while b < 2:
          try:
              c = a/b
          except ZeroDivisionError:
              b += 1
              c = "No es pot fer la divisió"
          finally:
              print(c)
              break
```

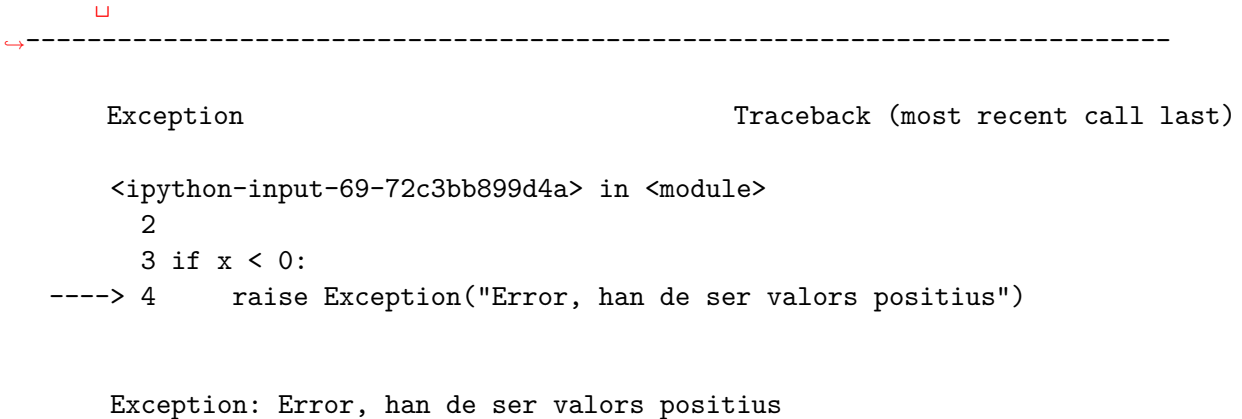
10.0

10.2 7.2 Raising Exceptions

Finalment, també es poden llançar excepcions específiques amb la instrucció `raise`. Aquesta instrucció permet definir quina mena d'error es genera i el text que s'ha d'imprimir a l'usuari.

```
[69]: x = -1

      if x < 0:
          raise Exception("Error, han de ser valors positius")
```



The image shows a Jupyter Notebook interface. At the top, there is a red square icon and a dashed line. Below this, the notebook displays a code cell with the following content:

```
Exception                                Traceback (most recent call last)

<ipython-input-69-72c3bb899d4a> in <module>
    2
    3 if x < 0:
----> 4     raise Exception("Error, han de ser valors positius")

Exception: Error, han de ser valors positius
```

Recomanem la lectura de la [documentació oficial](#) per acabar de fixar coneixements.

11 8.1 Instruccions importants

És molt important que a l'hora de lliurar el fitxer Notebook amb les vostres activitats us assegureu que:

1. Les vostres solucions siguin originals. Esperem no detectar-hi còpia directa entre estudiants.
2. Tot el codi estigui correctament documentat. El codi sense documentar equivaldrà a un 0.
3. El fitxer comprimit que lliureu és correcte (conté les activitats de la PAC que heu de lliurar).

Per fer el lliurament, heu d'anar a la carpeta del drive **Colab Notebooks**, clicant botó dret a la PAC en qüestió i fent **Download**. D'aquesta manera, us baixareu la carpeta de la PAC comprimida en zip. Aquest és l'arxiu que heu de pujar al campus virtual de l'assignatura.

12 9 Bibliografia

Us recomanem que consulteu els exemples següents del llibre *Learn Python 3 the hard way* de la bibliografia de l'assignatura, i que intenteu fer les activitats que s'hi proposen per acabar d'entendre i practicar els conceptes explicats en aquesta unitat.

Operacions lògiques: * Exercise 27. Memorizing Logic * Exercise 28. Boolean Practice

Condicionals: * Exercise 29. What If * Exercise 30. Else and If * Exercise 31. Making Decisions

Iteració: * Exercise 32. Loops and Lists * Exercise 33. While-Loops

13 Autors

- Autor original Brian Jiménez Garcia, 2016.
- Actualitzat per Cristina Pérez Solà, 2017 i 2019.
- Actualitzat per Joan Maynou Fernández, 2022.

```
<div style="width:0%;">&nbsp;</div>  
    
</div>
```